AI DD

AI-Powered M&A Due Diligence Platform

Complete Technical Study Guide

Version 1.0 Branch | Architecture + Implementation + Interview Prep

Backend: Python 3.12 + FastAPI + SQLAlchemy 2.0 (async) + Celery + Redis

AI/LLM: Groq API (Llama 3.3 70B) with mock fallback

Search: PostgreSQL Full-Text Search (tsvector + GIN)

OCR: pymupdf + pytesseract + Pillow

Frontend: React 19 + TypeScript + Tailwind CSS + React Query + Zustand

Infrastructure: Docker Compose (5 services) + uv package manager

11 Modules | 5 Docker Services | 60+ API Endpoints | 17+ Database Tables

April 2026 | *CONFIDENTIAL*

1. System Architecture

Tech Stack

Layer	Technology	Purpose
Backend	FastAPI + Python 3.12	Async REST API framework
ORM	SQLAlchemy 2.0 (async)	Database abstraction + migrations
Database	PostgreSQL 16	Primary data store + FTS + pgvector
Search	PostgreSQL tsvector + GIN	Full-text search (no vector DB)
AI/LLM	Groq API (Llama 3.3 70B)	Agent analysis + planning generation
Task Queue	Celery + Redis	Background OCR + agent processing
OCR	pymupdf + pytesseract	PDF/image text extraction
Frontend	React 19 + TypeScript	Single-page application
Styling	Tailwind CSS	Dark luxurious M&A-grade UI
State	React Query + Zustand	Server state + client state
Package Mgr	uv (Python) + npm (JS)	Fast dependency management
Containers	Docker Compose	5 services orchestration

Service Topology (Docker Compose)

```
PostgreSQL 16  (:5433) -- Primary database
Redis 7        (:6750) -- Celery broker + cache
FastAPI API    (:8000) -- Backend REST API
Celery Worker  (----)  -- Background task processing
React/Vite     (:5173) -- Frontend dev server
```

11 Registered Modules

Module	Status	Endpoints	Key Technology
Auth	100%	8	JWT + bcrypt + role hierarchy
Projects	100%	5	CRUD + member management
DMS	100%	5	Upload + OCR + FTS chunking
OCR	100%	internal	pymupdf + pytesseract + pandas
Agent	85%*	4	Groq API + FTS RAG + 4 agents
Planning	100%	6	5-phase workflow + Groq AI
Finance	100%	5	pandas + numpy variance
Report	100%	5	python-docx + .docx export
Audit	100%	2	Middleware + insert-only logs
Knowledge	10%	2	Models only, no logic
Self-Improve	10%	3	Models only, no logic

* Agent module returns mock findings when GROQ_API_KEY is not configured

2. Authentication & Authorization

JWT Token Flow

```
1. POST /auth/login  {email, password}
2. Server: bcrypt.verify(password, stored_hash)
3. Server: jwt.encode({sub: user_id, role: admin, type: access, exp: +60min})
4. Response: {access_token, refresh_token, token_type: bearer}
5. Client: localStorage.setItem('access_token', token)
6. Every request: Authorization: Bearer <token>
7. Server: jwt.decode() -> verify signature + expiry -> load User
```

5-Role Hierarchy (Spec 4.1)

Role	Projects	Documents	Analysis	Reports	Admin
Admin	Full CRUD	Full CRUD	Run+Review	Generate	Full
Lead Advisor	Create+Manage	Full CRUD	Run+Review	Generate	Audit only
Team Advisor	View assigned	Upload+View	Run+Review	View drafts	None
Seller	View own	Upload only	None	None	None
Buyer	View approved	View approved	None	Finalized only	None

Project-Level Access Control

```
# 3-tier permission dependencies:
project_reader    = any project member (or admin)
project_contributor = lead_advisor + team_advisor + seller
project_manager   = lead_advisor + team_advisor (not seller)

# Every project-scoped endpoint checks:
1. Is user authenticated? (JWT valid)
2. Is user admin? -> bypass all checks
3. Is user in ProjectMember table for this project? -> 403 if not
4. Does user's global role allow this action? -> 403 if not
```

Auto-Seeded Admin Users

```
On every startup, init_db() seeds:
  ayush@gmail.com / ayush@19 (admin, disclaimer accepted)
  aystar@gmail.com / aystar@19 (admin, disclaimer accepted)
New registrations default to 'buyer' role (least privileged)
```

3. Document Management + OCR

Upload Flow

```
User uploads file
  -> MIME validation (PDF, Word, Excel, CSV, images)
  -> Size check (<= 50MB)
  -> File saved to disk (uploads/)
  -> Document record: status='uploaded'
  -> BackgroundTask: _process_document()
       -> status='processing'
       -> extract_text(bytes, mime, filename)
       -> DocumentText created with full content
       -> status='ready'
       -> On error: status='failed'
```

OCR Engine (extractor.py)

Format	Library	Method
PDF (text)	pymupdf (fitz)	page.get_text() per page
PDF (scanned)	pymupdf + pytesseract	get_pixmap(300dpi) -> OCR
Word (.docx)	python-docx	Extract paragraphs
Excel (.xlsx)	openpyxl	Read all sheets, tab-separated
CSV/TSV	pandas	pd.read_csv() with delimiter
Images	pytesseract + Pillow	Image.open() -> image_to_string()
Plain text	stdlib	UTF-8 decode

Document Chunking for RAG

```
CHUNK_SIZE = 1500 chars (~1 paragraph)
CHUNK_OVERLAP = 300 chars (20%)

Document text -> sliding window -> chunks stored in document_chunks
Each chunk gets a tsvector search_vector (PostgreSQL FTS)
GIN index on search_vector for fast full-text queries

# Trigger auto-updates search_vector on INSERT/UPDATE:
search_vector = to_tsvector('english', chunk_text)
```

4. AI Agent System (RAG + Groq)

Architecture: Template Method Pattern

```
BaseAgent (abstract)
  |-- PlanningAgent  (audit scope, document gaps)
  |-- LegalAgent     (contracts, IP, litigation)
  |-- TaxAgent       (deferred tax, transfer pricing)
  |-- FinanceAgent   (EBITDA, revenue, working capital)

Each agent implements:
  agent_type       -> 'planning' | 'legal' | 'tax' | 'finance'
  _build_prompt()  -> (system_prompt, user_prompt)
  _mock_findings() -> hardcoded fallback data
```

RAG Pipeline (FTS-based, no vector embeddings)

```
1. CHUNKING: Document -> 1500-char chunks (300 overlap)
2. INDEXING: Each chunk -> tsvector + GIN index in PostgreSQL
3. RETRIEVAL: Agent query -> plainto_tsquery() -> ts_rank()
   -> Top 10 most relevant chunks returned
4. GENERATION: Chunks + system prompt -> Groq API
   Model: llama-3.3-70b-versatile
   Temperature: 0.1 (near-deterministic)
   Max tokens: 8192
   Response format: JSON {findings: [...]}
5. STORAGE: Findings saved to agent_findings table
```

Key Difference from Main Branch

Main branch used Google Vertex AI (Gemini 1.5 Pro) with pgvector cosine similarity for RAG. Version 1.0 switched to Groq API (Llama 3.3 70B) with PostgreSQL native FTS (tsvector). This eliminates the need for vector embeddings entirely -- simpler architecture, no external embedding service needed.

Anti-Hallucination Rules in Agent Prompts

- ONLY report findings with concrete evidence from documents
- Do NOT flag positive statements as risks
- Do NOT speculate or invent problems
- Quality over quantity: 3-5 findings maximum
- Every finding must reference specific document passage

Orchestrator Flow

```
run_analysis(run_id):
  1. Set run.status = 'running'
  2. Load all 'ready' documents for project
  3. For each doc: ensure_document_indexed() (chunk + FTS)
  4. For each workstream (sequential):
       agent = _AGENTS[workstream]
       findings = await agent.analyze(...)
       INSERT findings into agent_findings
  5. Set run.status = 'completed'
```

```
6. On error: status = 'failed', save error_message
```

5. Planning Module (The USP)

5-Phase Workflow

Phase 1: Basic Data	Advisor enters: company name, legal form, industry, employees, revenue, deal type
Phase 2: Risk Analysis	AI identifies 5-8 risk areas: legal-form risks, industry risks, deal-type risks
Phase 3: Dialog	AI asks 5-8 targeted questions: pending litigation? tax structures? key contracts?
Phase 4: Audit Plan	Structured plan: scope, objectives, workstreams, timeline, resources. Human approves.
Phase 5: Request List	15 items across Legal/Tax/Finance with status + priority tracking

AI Integration

```
# Each phase transition calls Groq API:
async _call_groq(system_prompt, user_prompt) -> dict
  Model: llama-3.3-70b-versatile
  Temperature: 0.2
  Max tokens: 4096
  Format: JSON object

# If no Groq key: returns structured mock data
# Mock data is realistic (not placeholder)
```

Request List Generation

15 standard DD items generated from templates, prioritized by identified risks:

- Legal: Corporate structure chart, shareholder agreements, material contracts, employment contracts, IP register
- Tax: Tax returns (3-5 years), assessment notices, loss carryforwards, transfer pricing docs
- Finance: Audited statements (3-5 years), management accounts, EBITDA bridge, working capital analysis

6. Financial Analysis Module

Data Pipeline

```
Upload Excel/CSV
  -> pandas.read_csv() or read_excel()
  -> Clean column names, replace NaN
  -> Store raw_data as JSON in financial_datasets
  -> Parse into financial_line_items

Variance Analysis:
  -> Load all datasets for project
  -> Extract numeric columns via pandas
  -> Compute: mean, std, min, max, variance_pct
  -> Flag 'significant' if variance_pct > 20%
  -> Generate follow-up queries for flagged items
```

Models

Table	Key Columns	Purpose
financial_datasets	raw_data (JSON), chart_of_accounts	One per uploaded file
financial_line_items	account, period, amount	Individual data points
variance_analyses	results (JSON), queries (JSON)	Computed analysis output

7. Report Generation

3 Report Types

Type	Content	Format
Detailed Workstream	All findings per workstream, grouped by category	Word .docx
Executive Summary	Critical + high findings only, editable before export	Word .docx
Consolidated	Cross-workstream summary, grouped by agent type	Word .docx

Report Generation Pipeline

```
POST /reports/generate:
  1. Fetch ALL completed AgentRun rows for project
  2. Load AgentFinding rows from all completed runs
  3. Deduplicate findings by title (keep earliest)
  4. Filter by workstream (if detailed report)
  5. Structure content by report type
  6. Append AI Disclaimer to all reports
  7. Return Report with JSON content

POST /reports/{id}/finalize:
  1. python-docx creates Word document
  2. Adds title, disclaimer, metadata
  3. Iterates content sections
  4. Saves to uploads/reports/{uuid}.docx
  5. Locks report (is_finalized = True)
```

AI Disclaimer (Mandatory)

> *Notice: This system uses Artificial Intelligence to support the due diligence review. AI-generated results may be inaccurate, incomplete, or misleading. Responsibility for audit results lies exclusively with the human reviewer.*

8. Audit Trail (GDPR Compliance)

Middleware-Based Logging

```
class AuditMiddleware:
  # Runs on EVERY API request
  # Skips: /health, /docs, /redoc, /favicon.ico
  # For each request:
  #   1. Extract method, path, status code
  #   2. Capture IP address + user-agent
  #   3. INSERT into audit_logs (never UPDATE/DELETE)
  #   4. Log is immutable (GDPR tamper-proof)
```

Audit Log Schema

Column	Type	Purpose
user_id	UUID	Who performed the action
action	Enum	login, document_uploaded, agent_run_started, etc.
resource_type	String	document, project, user, report
resource_id	String	UUID of affected resource
ip_address	String	Client IP (for access tracking)
created_at	DateTime	Immutable timestamp

9. Frontend Architecture

Route Map (14 routes)

```
PUBLIC:
  /              -> LandingPage
  /login         -> LoginPage
  /register      -> RegisterPage
  /disclaimer    -> DisclaimerPage

PROTECTED (AuthGuard):
  /dashboard                       -> DashboardPage
  /projects                        -> ProjectsPage
  /projects/:id/documents          -> DocumentsPage
  /projects/:id/planning           -> PlanningPage
  /projects/:id/analysis           -> AgentsPage
  /projects/:id/analysis/:runId    -> AgentRunPage
  /projects/:id/finance            -> FinancePage
  /projects/:id/reports            -> ReportsPage
  /projects/:id/red-flags          -> RedFlagsPage
  /projects/:id/settings           -> ProjectSettingsPage
  /admin/users                     -> AdminUsersPage
  /audit                           -> AuditLogsPage
  /settings                        -> SettingsPage
```

Permission Hook (usePermissions)

```
const perms = usePermissions();
// Returns:
perms.canCreateProject     // admin, lead_advisor
perms.canUploadDocuments   // admin, lead, team, seller
perms.canRunAnalysis       // admin, lead, team
perms.canManageReports     // admin, lead, team
perms.onlyFinalizedReports // buyer only
perms.isReadOnly           // buyer only
perms.canViewAudit         // admin, lead_advisor
```

State Management

Tool	Manages	Pattern
React Query	Server state (API data)	Cache + refetch + polling
Zustand	Client state (auth user)	Global store + localStorage
Axios interceptor	Auth headers + 401 redirect	Request/response middleware

10. Deployment Architecture

Production Target: Google Cloud Run (europe-west3)

Service	GCP Product	Est. Cost/month
API	Cloud Run	EUR 20-50
Frontend	Cloud Run (static)	EUR 5-10
Database	Cloud SQL (PostgreSQL 16)	EUR 30-80
Redis	Memorystore	EUR 30-50
Documents	Cloud Storage	EUR 5-20
AI	Groq API (external)	Pay per token
CI/CD	Cloud Build	Free tier
SSL	Managed certificates	Free
TOTAL		EUR 100-250

Security Checklist

- HTTPS via Cloud Load Balancer + managed SSL
- Secrets in GCP Secret Manager (not env vars)
- CORS restricted to production domain only
- JWT secret: 64-char random string
- Database: Cloud SQL IAM auth
- Rate limiting via Cloud Armor
- Automated backups (Cloud SQL)
- EU data residency (europe-west3 only)

11. What's Missing / Needs Improvement

Critical Gaps

Issue	Impact	Fix Effort
Groq fallback is silent	Users get mock data unknowingly	2 hours
BackgroundTasks not durable	Tasks lost on restart	4 hours (Celery)
No token blacklist/logout	Tokens valid until expiry	3 hours
No 2FA/TOTP	Single-factor auth only	4 hours
No GDPR user deletion	Compliance gap	3 hours
No password reset	Users locked out	3 hours
Knowledge module empty	No cross-project learning	8 hours
Self-improvement empty	No usage pattern tracking	8 hours
No Alembic migrations	Schema changes risky	2 hours
Zero automated tests	No regression safety	Ongoing

Features in Main Branch (Not in v1.0)

- Token blacklist + proper logout
- 2FA/TOTP authentication
- Password reset flow
- GDPR user deletion (Art. 17)
- Settings page (password change + 2FA)
- Document versioning (version_number + parent_doc_id)
- Full-text search on documents (tsvector)
- AI auto-tagging (10 document categories)
- 7-state document lifecycle
- SKR03/SKR04 chart detection + German number parsing
- 6 Alembic migration files

12. Numbers to Remember

Metric	Value	Context
Modules	11	All registered in main.py
API Endpoints	~60+	Across all modules
Database Tables	17+	Auth, Projects, DMS, Agent, Finance, Planning, Report, Audit,
Docker Services	5	PostgreSQL, Redis, API, Celery, Web
User Roles	5	Admin, Lead Advisor, Team Advisor, Seller, Buyer
Agent Types	4	Planning, Legal, Tax, Finance
Planning Phases	5	Basic Data -> Risk -> Dialog -> Plan -> Request List
Report Types	3	Detailed, Executive Summary, Consolidated
Request List Items	15	Standard DD checklist across 3 workstreams
Chunk Size	1500 chars	With 300 char overlap
FTS Top-K	10	Chunks per agent query
LLM Model	llama-3.3-70b	Via Groq API
LLM Temperature	0.1 (agents), 0.2 (planning)	Low for consistency
JWT Access Expiry	60 minutes	HS256 algorithm
JWT Refresh Expiry	30 days	For session persistence
Max File Size	50 MB	Document upload limit
Variance Threshold	>20%	Flagged as 'significant'
Password Hash	bcrypt	Deliberately slow (~100ms)

Overall Completion

```
Auth + Projects     [====================] 100%
DMS + OCR           [====================] 100%
AI Agents           [=================   ]  85% (needs Groq key)
Planning            [====================] 100%
Finance             [====================] 100%
Reports             [====================] 100%
Audit Trail         [====================] 100%
Knowledge           [==                  ]  10% (scaffolded)
Self-Improvement    [==                  ]  10% (scaffolded)
Deployment/CI       [                    ]   0%

OVERALL: ~75% of specification complete
```